

中山大学计算机学院

人工智能

本科生实验报告

(2022 学年春季学期)

课程名称: Artificial Intelligence

教学班级	人工智能 (饶洋辉老师)	专业 (方向)	23 级地理信息科学 (辅修)
学号	23306103	姓名	赵义凯

一、实验题目

使用 A*算法和 IDA*算法解决 15puzzle 问题

二、实验内容

1. 算法原理

- a) **A* 算法:** A* 算法是一种启发式搜索算法, 结合了实际代价 (从起始状态到当前状态的代价, 记为 g) 和启发式估计代价 (当前状态到目标状态的估计代价, 记为 h), 通过计算 $f = g + h$ 来评估每个状态的优先级。在 15-Puzzle 问题中, 实际代价 g 通常是移动的步数, 启发式函数 h 选择曼哈顿距离。曼哈顿距离计算每个数字当前位置与目标位置在行列方向上的距离之和, 能有效反映当前状态与目标状态的差异程度。算法使用优先队列 (最小堆) 存储待扩展状态, 每次从优先队列中取出 f 值最小的状态进行扩展, 直到找到目标状态或队列为空。
- b) **IDA* 算法:** IDA 算法是一种迭代加深的 A 算法。它首先设置一个初始阈值, 该阈值通常为初始状态的启发式函数值 (如曼哈顿距离)。在每次迭代中, 使用深度优先搜索 (DFS) 进行搜索, 搜索过程中计算每个状态的 f 值 ($f = g + h$), 当 f 值超过阈值时, 回溯并放弃该路径。如果在当前阈值下找到目标状态, 则返回解; 否则, 提高阈值并重新进行搜索, 直到找到目标状态。这种迭代加深的方式在内存使用上较为高效, 尤其适用于状态空间较大的问题。

由于我的电脑自身算力有限, 起初在尝试随机生成一个可解的十五数码问题时, 程序经常会寻找很多步也找不到解, 于是我限制步数在 20 步内的初始状态, 利用 Lab3 参考文档中

的随机生成初始状态的代码，再利用计算曼哈顿距离函数，只有在初始状态的曼哈顿距离在 20 以内才进行后续的步骤。然后通过初始状态的曼哈顿距离去选择对应的搜索算法，15 步以内使用 A* 算法，其余选择 IDA* 算法。

2. 伪代码

A* 算法伪代码:

```
function AStar(start_state, goal_state):
    open_list ← 优先队列 (最小堆)
    visited ← 集合
    open_list.add((h(start_state), 0, start_state, []))
    visited.add(start_state)
    while open_list is not empty:
        f, g, current_state, path ← open_list.pop()
        if current_state == goal_state:
            return path
        for move in get_possible_moves(current_state):
            new_state ← apply_move(current_state, move)
            if new_state not in visited:
                visited.add(new_state)
                new_path ← path + [move]
                open_list.add((g + 1 + h(new_state), g + 1, new_state, new_path))
    return null
```

IDA* 算法伪代码:

```
function IDAStar(start_state, goal_state):
    threshold ← h(start_state)
    path ← [start_state]
    while true:
        result ← recursive_search(path, 0, threshold)
        if result is list:
            return result
        threshold ← result
    function recursive_search(path, g, threshold):
        current_state ← path[-1]
        f ← g + h(current_state)
        if f > threshold:
            return f
        if current_state == goal_state:
            return path
        min_threshold ← 无穷大
        for move in get_possible_moves(current_state):
            new_state ← apply_move(current_state, move)
            if new_state not in path:
                path.append(new_state)
                result ← recursive_search(path, g + 1, threshold)
                if result is list:
                    return result
                min_threshold ← min(min_threshold, result)
            path.pop()
        return min_threshold
```

3. 关键代码展示 (带注释)



```
def is_solvable(puzzle):
    # 计算逆序对的个数
    inversions = 0
    for i in range(16):
        for j in range(i + 1, 16):
            if puzzle[i] and puzzle[j] and puzzle[i] > puzzle[j]:
                inversions += 1
    # 找到空格的位置
    empty_row = 4 - (puzzle.index(0) // 4)
    # 判断是否可解
    return (inversions % 2 == 0) == (empty_row % 2 == 1)

def generate_random_puzzle():
    puzzle = list(range(16))
    random.shuffle(puzzle)
    while not is_solvable(puzzle):
        random.shuffle(puzzle)
    return puzzle

def convert_id_to_2d(puzzle_id):
    return [puzzle_id[i:i + 4] for i in range(0, 16, 4)]

# 目标状态，也就是谜题最终要达到的样子
GOAL_STATE = [[1, 2, 3, 4],
               [5, 6, 7, 8],
               [9, 10, 11, 12],
               [13, 14, 15, 0]]

# 生成随机初始状态，确保20步内可解
while True:
    random_puzzle_id = generate_random_puzzle()
    INITIAL_STATE = convert_id_to_2d(random_puzzle_id)
    start_manhattan_distance = sum(
        abs((value - 1) // 4 - i) + abs((value - 1) % 4 - j)
        for i in range(4)
        for j in range(4)
        for value in INITIAL_STATE[i][j])
    if value != 0
    )
    if start_manhattan_distance < 20:
        break

# 将二维的状态数组转换为一维元组，方便后续操作和比较
def convert_state_to_tuple(state):
    return tuple(element for sub_array in state for element in sub_array)

# 找到空白方块（值为0）在状态数组中的位置
def locate_zero(state):
    for i in range(4):
        for j in range(4):
            if state[i][j] == 0:
                return i, j

# 根据当前状态，生成所有可能的移动方向
def generate_possible_moves(state):
    possible_moves = []
    zero_row, zero_col = locate_zero(state)
    # 空白方块向上移动
    if zero_row > 0:
        possible_moves.append("up")
    # 空白方块向下移动
    if zero_row < 3:
        possible_moves.append("down")
    # 空白方块向左移动
    if zero_col > 0:
        possible_moves.append("left")
    # 空白方块向右移动
    if zero_col < 3:
        possible_moves.append("right")
    return possible_moves

# 对当前状态应用指定的移动，返回移动后的新状态
def apply_movement(state, move):
    zero_row, zero_col = locate_zero(state)
    # 复制当前状态，避免修改原始状态
    new_state = [row[:] for row in state]
    if move == "up":
        new_state[zero_row][zero_col], new_state[zero_row - 1][zero_col] = new_state[zero_row - 1][zero_col], new_state[zero_row][zero_col]
    elif move == "down":
        new_state[zero_row][zero_col], new_state[zero_row + 1][zero_col] = new_state[zero_row + 1][zero_col], new_state[zero_row][zero_col]
    elif move == "left":
        new_state[zero_row][zero_col], new_state[zero_row][zero_col - 1] = new_state[zero_row][zero_col - 1], new_state[zero_row][zero_col]
    elif move == "right":
        new_state[zero_row][zero_col], new_state[zero_row][zero_col + 1] = new_state[zero_row][zero_col + 1], new_state[zero_row][zero_col]
    return new_state

# 启发式函数，计算当前状态到目标状态的曼哈顿距离
def calculate_manhattan_distance(state):
    total_distance = 0
    for i in range(4):
        for j in range(4):
            value = state[i][j]
            if value != 0:
                # 计算目标位置
                target_row = (value - 1) // 4
                target_col = (value - 1) % 4
                total_distance += abs(target_row - i) + abs(target_col - j)
    return total_distance

# 使用 A* 算法来解决 15 - 谜题
def a_star_search(start_state):
    start_state_tuple = convert_state_to_tuple(start_state)
    goal_state_tuple = convert_state_to_tuple(GOAL_STATE)
    # A* 算法的优先队列，用最小堆实现
    open_list = []
    heapq.heappush(open_list, (calculate_manhattan_distance(start_state), 0, start_state_tuple, []))
    # 记录已经访问过的状态
    visited = set()
    visited.add(start_state_tuple)
    while open_list:
        # 取出 f 值 (g + h) 最小的状态
        f, g, current_state_tuple, path = heapq.heappop(open_list)
        current_state = [list(current_state_tuple[i:i + 4]) for i in range(0, len(current_state_tuple), 4)]
        # 检查是否到达目标状态
        if current_state_tuple == goal_state_tuple:
            return path
        # 生成所有可能的移动
        for move in generate_possible_moves(current_state):
            new_state = apply_movement(current_state, move)
            new_state_tuple = convert_state_to_tuple(new_state)
            # 如果新状态未被访问过
            if new_state_tuple not in visited:
                visited.add(new_state_tuple)
                new_path = path + [move]
                heapq.heappush(open_list, (g + 1 + calculate_manhattan_distance(new_state), g + 1, new_state_tuple, new_path))
```



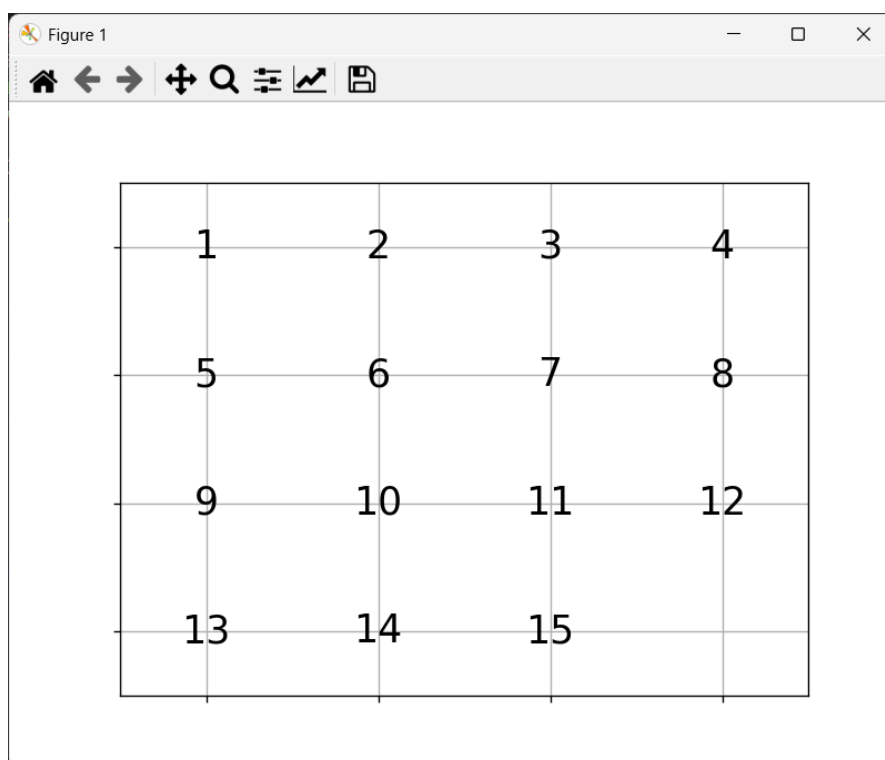
```
# 使用 IDA* 算法来解决 15 - 谜题
def ida_star_search(start_state):
    start_state_tuple = convert_state_to_tuple(start_state)
    goal_state_tuple = convert_state_to_tuple(GOAL_STATE)
    # 初始阈值为初始状态的曼哈顿距离
    threshold = calculate_manhattan_distance(start_state)
    def recursive_search(path, g, threshold):
        current_state_tuple = path[-1]
        current_state = [list(current_state_tuple[i:i + 4]) for i in range(0, len(current_state_tuple), 4)]
        f = g + calculate_manhattan_distance(current_state)
        # 如果 f 值超过阈值, 返回 f 值
        if f > threshold:
            return f
        # 如果到达目标状态, 返回路径
        if current_state_tuple == goal_state_tuple:
            moves = []
            for i in range(len(path) - 1):
                current = [list(path[i][j:j + 4]) for j in range(0, len(path[i]), 4)]
                next_state = [list(path[i + 1][j:j + 4]) for j in range(0, len(path[i + 1]), 4)]
                for move in generate_possible_moves(current):
                    if apply_movement(current, move) == next_state:
                        moves.append(move)
                        break
            return moves
        min_threshold = float('inf')
        for move in generate_possible_moves(current_state):
            new_state = apply_movement(current_state, move)
            new_state_tuple = convert_state_to_tuple(new_state)
            # 如果新状态未在路径中
            if new_state_tuple not in path:
                path.append(new_state_tuple)
                result = recursive_search(path, g + 1, threshold)
                if isinstance(result, list):
                    return result
                min_threshold = min(min_threshold, result)
            path.pop()
        return min_threshold
    # 初始化搜索
    path = [start_state_tuple]
    while True:
        result = recursive_search(path, 0, threshold)
        if isinstance(result, list):
            return result
        # 更新阈值
        threshold = result
# 根据初始状态和用户选择, 决定使用 A* 还是 IDA* 算法
def select_algorithm(start_state, use_ida_star=False):
    if use_ida_star:
        print("选择 IDA* 算法")
        return ida_star_search(start_state)
    else:
        start_manhattan_distance = calculate_manhattan_distance(start_state)
        print(f"初始曼哈顿距离: {start_manhattan_distance}")
        # 如果初始曼哈顿距离小于15, 使用 A* 算法
        if start_manhattan_distance < 15:
            print("选择 A* 算法")
            return a_star_search(start_state)
        else:
            print("选择 IDA* 算法")
            return ida_star_search(start_state)
```

4. 创新点&优化（如果有）

- a) **优化算法选择：**通过计算初始状态的曼哈顿距离，根据距离大小自动选择 A 算法或 IDA 算法。当曼哈顿距离小于 20 时，选择 A 算法，因为此时状态空间相对较小，A 算法的全局搜索优势能快速找到解；当曼哈顿距离大于等于 20 时，选择 IDA* 算法，利用其迭代加深的特性，在较大状态空间中减少内存消耗，提高搜索效率。
- b) **可视化解题过程：**使用 matplotlib 库的 FuncAnimation 函数实现解题过程的可视化。将每一步的状态存储在列表中，通过动画的形式展示从初始状态到目标状态的变化过程，使解题过程更加直观，便于理解和分析算法的执行情况。

```
def visualize_solution(solution, initial_state):
    states = [initial_state]
    current_state = initial_state
    # 依次应用每一步移动，记录所有状态
    for move in solution:
        current_state = apply_movement(current_state, move)
        states.append(current_state)
    fig, ax = plt.subplots()
    # 设置坐标轴范围
    ax.set_xlim(-0.5, 3.5)
    ax.set_ylim(-0.5, 3.5)
    # 设置坐标轴刻度
    ax.set_xticks(np.arange(0, 4, 1))
    ax.set_yticks(np.arange(0, 4, 1))
    # 隐藏刻度标签
    ax.set_xticklabels([])
    ax.set_yticklabels([])
    # 显示网格线
    ax.grid(True)
    def update(frame):
        ax.clear()
        ax.set_xlim(-0.5, 3.5)
        ax.set_ylim(-0.5, 3.5)
        ax.set_xticks(np.arange(0, 4, 1))
        ax.set_yticks(np.arange(0, 4, 1))
        ax.set_xticklabels([])
        ax.set_yticklabels([])
        ax.grid(True)
        state = states[frame]
        for i in range(4):
            for j in range(4):
                value = state[i][j]
                if value != 0:
                    ax.text(j, 3 - i, str(value), ha='center', va='center', fontsize=20)
    # 创建动画
    ani = FuncAnimation(fig, update, frames=len(states), interval=500, repeat=False)
    plt.show()
```

可视化最终结果：



三、 实验结果及分析

1. 实验结果展示示例（可图可表可文字，尽量可视化）

```
PS D:\大二下\人工智能资料\实验课\23306103_赵义凯_实验3> python -u "d:\大二下\人工智能资料\实验课\23306103_赵义凯_实验3\main.py"
初始曼哈顿距离: 10
选择 A* 算法
解决方案: ['up', 'up', 'left', 'left', 'down', 'right', 'right', 'up', 'up', 'left', 'down', 'down', 'right', 'down', 'left', 'up', 'left', 'up', 'left', 'up',
'right', 'right', 'right', 'down', 'down', 'down']
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\大二下\人工智能资料\实验课> cd "d:\大二下\人工智能资料\实验课\23306103_赵义凯_实验3"
PS D:\大二下\人工智能资料\实验课\23306103_赵义凯_实验3> python -u "d:\大二下\人工智能资料\实验课\23306103_赵义凯_实验3\main.py"
初始曼哈顿距离: 19
选择 IDA* 算法
解决方案: ['down', 'left', 'down', 'down', 'left', 'up', 'up', 'up', 'right', 'right', 'down', 'down', 'down', 'left', 'up', 'up', 'up', 'right', 'down', 'left',
'left', 'down', 'right', 'right', 'up', 'up', 'left', 'left', 'down', 'left', 'down', 'right', 'up', 'right', 'down', 'right', 'down']
```

四、 思考题

1. 算法优化策略的拓展：实验中通过根据初始状态曼哈顿距离选择算法来优化求解过程。思考是否还有其他优化策略可以进一步提升算法效率，比



如在搜索过程中动态调整启发式函数，或者对已访问状态的存储结构进行优化，以减少搜索时间和内存消耗。

五、 参考资料

[1]Lab3 参考文档

[2]CSDN: [算法——对比 A*算法与 IDA*算法](#)